COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithms | **SECTION:** | CPE21S3 |
| **DATE PERFORMED:** | July 28, 2026 | **DATE SUBMITTED:** | Aug 4, 2026 |
| **NAME:** | Dela Cruz, Jared Isaiah P. | **INSTRUCTOR:** | Engr. Jimlord Quejado |

**ACTIVITY NO. 4.1** **Array-Based vs. STL Stack Implementations**

**1. PROCEDURE OUTPUT**

## Tasks & Intended Learning Outcomes (ILOs):

- **ILO A:** Implement a stack using the C++ STL (std::stack).

**OUTPUT/PROGRAM:**

```cpp
#include <iostream>
#include <stack>


int main(){
    std::stack<int> newStack;

    std::cout<<"Is the stack empty? "<< newStack.empty() <<std::endl;

    newStack.push(3);
    std::cout<<"Top of Stack: "<< newStack.top() <<std::endl;

    newStack.push(2);
    std::cout<<"Top of Stack: "<< newStack.top() <<std::endl;

    newStack.push(1);
    std::cout<<"Top of Stack: "<< newStack.top() <<std::endl;

    std::cout<<"Is the stack empty? "<< newStack.empty() <<std::endl;
    newStack.push(6);
    newStack.push(7);
    newStack.push(8);

    std::cout<<"Size of the Stack is: "<< newStack.size() <<std::endl;

     if (newStack.size() >= 6){
        int nextValue = newStack.top();
        newStack.pop();

        int topValue = newStack.top();
        newStack.pop();

        newStack.push(topValue);
        newStack.push(nextValue);

        std::cout << "New Value: " <<newStack.top() <<std::endl;
    }
    return 0;
}
```

**ANALYSIS:** This program showcases an array-based stack. It basically visualizes how we can push values into the stack and the program will be able to determine which value is at the top of the stack, if the stack is empty, or

what size the stack currently is. As an extra challenge, I added a value swap function where it takes the two top stack values and switches them, giving a new value afterwards.

- **ILO B.1:** Implement a stack using a Static Array.

**OUTPUT/PROGRAM:**

HOA4p1 > ILO B1 > main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>

const size_t maxCap = 100;
int stack[maxCap];
int top = -1, newData, elements;


bool isEmpty();
bool isFull();
void push();
void pop();
void Top();
void displayAll();

int main(){
    int choice;
    std::cout<<"Enter the number of maximum elements for the new stack (max 100)";
    std::cin >> elements;

    while(true){
        std::cout << "Stack Operations: " <<std::endl;
        std::cout << "1.PUSH \n2.POP \n3.TOP \n4.isEmpty \n5.isFull \n6.Display Stack \n7.Exit" <<std::endl;
        std::cin >> choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout << isEmpty() <<std::endl;
            break;
            case 5: std::cout << isFull() <<std::endl;
            break;
            case 6: displayAll();
            break;
            case 7: std::cout << "Exiting Program" <<std::endl;
                exit(0);
            default: std::cout <<"Invalid Choice" << std::endl;
            break;
        }
    }
    return 0;
}

bool isEmpty(){
```

```
49      if(top == -1){
50          std::cout<<"Stack Underflow";
51          return true;
52      }
53      return false;
54  }
55
56  bool isFull(){
57      if(top >= elements){
58          std::cout<<"Stack OVerflow" << std::endl;
59          return true;
60      }
61      return false;
62  }
63
64  void push(){
65      if(isFull()) return;
66
67      std::cout<<"What number do you want to add on the stack? ";
68      std::cin >> newData;
69      stack[++top] = newData;
70  }
71
72  void pop(){
73      if(isEmpty()) return;
74
75      std::cout<<"Popping: "<< stack[top]<<std::endl;
76
77      top--;
78
79  }
80
81  void Top(){
82      if(isEmpty()) return;
83
84      std::cout<<"The Top Element is: "<<stack[top]<<std::endl;
85
86  }
87
88  void displayAll(){
89      if(isEmpty()) return;
90
91      for(int i; i <= elements; i++){
92          std::cout<<stack [elements - i]<<std::endl;
93      }
```

**ANALYSIS:** For this program, we followed the given procedure to showcase the implementation of stacks using a static array. With this, it is shown to us the different functions stored in the stack and make changes with it based

on the user input. I added a Display Stack function so that it can visualize the contents and changes made to the stack in real time for easier tracking. I also added an exit function that terminates the program when the user is done with using the program.

**ILO B.2**

**OUTPUT/PROGRAM:**

HOA4p1 > SUPPLEMENTARY4.1 > main.cpp > main()

```cpp
#include <iostream>
#include "listStack.h"

int main() {
    std::cout << "-------- Pushing 'I' 'E' 'E' 'E' -------- \n";

    push('I');
    push('E');
    push('E');
    push('E');

    Top<char>();
    std::cout << "\n";

    std::cout << "-------- popping 'I' 'E' 'E' 'E' -------- \n";

    while (!isEmpty<char>()) {
        std::cout << "Popping: " << pop<char>() << std::endl;
    }

    pop<char>();

    return 0;
}
```

HOA4p1 > ILO B2 > C listStack.h > Top<T>()

```cpp
#ifndef LIST_STACK_H
#define LIST_STACK_H

template <typename T>
class Node{
    public:
    T data{};
    Node<T>* next = nullptr;
};

template <typename T>
inline Node<T>* head = nullptr;

template <typename T>
void push(T newData){
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = head<T>;
    head<T> = newNode;
}

template <typename T>
bool isEmpty(){
    return head<T> == nullptr;
}

template <typename T>
T pop (){
    if(isEmpty<T>()){
        std::cout<<"Stack Underflow"<<std::endl;
        return T{};
    }

    Node<T>* temp = head<T>;
    T tempVal = temp->data;
    head<T> = head<T>->next;
    delete temp;
    return tempVal;
}

template <typename T>
void Top(){
    if(isEmpty<T>()) return;
    std::cout<<"The top is: "<<head<T>->data;
}
#endif
```

**OBSERVATIONS:**

**SUPPLEMENTARY ACTIVITY**

**OUTPUT/PROGRAM:**

| EXPRESSION | VALID? | OUTPUT | ANALYSIS |
|---|---|---|---|
| (A+B)+(C-D) | YES | Enter an expression: (A+B)+(C-D)<br>The expression is Balanced. | For the given expression (A+B)+(C-D), the checkBalanced function goes through each character in the string and tests it based on the stack functions. As soon as the function reaches ( in the string, isOpening(ch) will be true and push it onto the stack. As soon as ) is encountered, isClosing(ch) will call pop<char>(), which takes the top open symbol from the stack. The isMatching function will compare the popped open symbol '(' with the closing symbol ')' and will return true for both. Since all symbols have been matched and there are no unclosed delimiters left after looping, the stack will become empty and allow the function to skip all error checking and return true. |
| ((A+B)+(C-D) | NO | Enter a sample expression: ((A+B)+(C-D)<br>Error. There are mismatches with the opening symbols.<br>The given sample expression is Unbalanced. | For the next input ((A+B)+(C-D)), the process starts by first removing all existing data in the stack and then evaluating each character in the expression. In this case, the three opening parenthesis symbols ( are stored into the stack while only two closing symbols ) are read to perform a pop<char>() command. Once the for loop finishes processing the string, |

| | | | |
|---|---|---|---|
| | | | the function checks if(!isEmpty<char>()) whether there are still unmatched symbols left in the stack. Since there is an additional parenthesis symbol ( that has not been closed or popped out, the stack is not empty. This meets the condition that makes the program display "Error. There are mismatches with the opening symbols." and returns false. |
| ((A+B)+[C-D]) | YES | Enter a sample expression: ((A+B)+[C-D])<br>The given sample expression is Balanced. | For the input ((A+B)+[C-D]), the program, during the iteration of the string in the loop, isOpening(ch) function pushes the two (, pops it when the first ) is encountered, pushes [ on encountering [, pops [ on encountering ], and pops the other ( on encountering ). At every closing delimiter, isMatching(open, ch) ensures that the popped opening delimiter is matched with the current closing delimiter. After the iteration of the string, it checks that the stack is completely empty using isEmpty<char>(). Since there are no errors, checkBalanced returns true. |
| ((A+B]+[C-D]} | NO | Enter a sample expression: ((A+B]+[C-D]}<br>Error: '(' mismatch detected ']'.<br>The given sample expression is Unbalanced. | For the expression ((A+B]+[C-D]}, the program analyzes each character one by one until it finds a mismatch in the types of brackets. Two opening brackets ( get pushed to the stack. But once the program comes to the closing bracket ], the function isClosing(ch) calls the pop<char>() function, which gets the latest opening bracket, that is, '('. Now, if we pass the two brackets to the function |

| | | | |
|---|---|---|---|
| | | | isMatching('(', ']'), we will get false as a return value because the round bracket cannot match the square bracket. This meets our criteria for if (!isMatching(open, ch)), and thus, it prints "Error: '(' mismatch detected ']'" and returns false right away. |

**MAIN PROGRAM:**

**main.cpp/SUPPL2.cpp**

```cpp
#include <iostream>
#include <string>
#include "SUPPL_listStack.h"

bool isOpening(char ch){
    return ch == '(' || ch == '{' || ch == '[';
}

bool isClosing(char ch){
    return ch == ')' || ch == '}' || ch == ']';
}

bool isMatching(char open, char close){
//returns true if the symbols match, otherwise false
    if(open == '(' && close == ')')
        return true;
    if(open == '{' && close == '}')
        return true;
    if(open == '[' && close == ']')
        return true;
    return false;
}

bool checkBalanced(std::string expression){
    // Clear the stack to avoid errors from previous expression
    while(!isEmpty<char>())
        pop<char>();

    for(char ch : expression){
        //Push to stack if opening symbol
        if(isOpening(ch)){
            push<char>(ch);
        }
        //Now for closing symbols
        else if(isClosing(ch)){
            // If the stack is empty, there is no matching opening symbol
            if(isEmpty<char>()){
                std::cout << "Error with Closing symbol '" << ch << "' mismatch with the opening symbols.\n";
                return false;
            }
            //Start popping
            char open = pop<char>();
            // Compare the opening and closing symbols
            if(!isMatching(open, ch)){
                std::cout << "Error: '" << open << "' mismatch detected '" << ch << "'.\n";
                return false;
            }
        }
    }
    // Check if there are still opening symbols left in the stack
    if(!isEmpty<char>()){
        std::cout << "Error. There are mismatches with the opening symbols.\n";
        return false;
    }
    // If no errors were found, the expression is balanced
    return true;
}

int main(){
```

```
    std::string expression;

    std::cout << "Enter a sample expression: ";
    getline(std::cin, expression);

    if(checkBalanced(expression))
        std::cout << "The given sample expression is Balanced.\n";
    else
        std::cout << "The given sample expression is Unbalanced.\n'

    return 0;
}
```

---

**ANALYSIS:**

- This showcases the main program proper. Within are the helper functions isOpening, isClosing, and isMatching describing the conditions for acceptable delimiters: parentheses, brackets, and braces, and the logic for matching these characters. In the checkBalanced function, the program deletes any remaining nodes in the stack template before going through each character in the inputted string: adding an opening character to the stack, popping off from the stack when a closing character appears, and ensuring that the popped character correctly matches the closing character. When the stack is empty but a closing character appears, or the isMatching returns false, the program throws an error message and returns false. At the end of checking all characters in the expression, the program checks if there are still unmatched characters in the stack and throws an unclosed error if non-empty, or returns true to the main() function.

**Header**

```cpp
#ifndef LIST_STACK_H
#define LIST_STACK_H

#include <iostream>

template <typename T>
class Node{
public:
    T data{};
    Node<T>* next = nullptr;
};

template <typename T>
Node<T>* head = nullptr;

template <typename T>
void push(T newData){
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = head<T>;
    head<T> = newNode;
}

template <typename T>
bool isEmpty(){
    return head<T> == nullptr;
}

template <typename T>
T pop(){
    if(isEmpty<T>()){
        return T{};
    }

    Node<T>* temp = head<T>;
    T tempVal = temp->data;
    head<T> = head<T>->next;

    delete temp;
    return tempVal;
}

template <typename T>
T Top(){
    if(isEmpty<T>())
        return T{};

    return head<T>->data;
}

#endif
```

**ANALYSIS:**

- The header showcases a generic implementation of a Stack data structure using a singly linked list template in C++. It defines a class template Node<T> holding a data member of type T and a pointer next pointing to the subsequent node, alongside a global template pointer head<T> initialized to nullptr representing the top of the stack. The push function handles element insertion by dynamically allocating a new Node<T>, setting its payload, pointing its next member to the current head<T>, and reassigning head<T> to this new node, effectively implementing a Last-In, First-Out prepending operation. Memory cleanup and element removal are managed by pop, which verifies if the stack is non-empty, saves the top node's payload, advances head<T> to head<T>->next, deallocates the original top node via delete, and returns the stored value (or a default-constructed value T{} if empty). Auxiliary utility functions isEmpty and Top allow safe inspection of the stack's state without modifying its layout by checking for nullptr and returning the head node's data respectively.

**CONCLUSION:**

- Overall, the module helped showcase that a simple linear data structure can be utilized in order to resolve problems with specifications just by using stacks. The program has been able to record and match each opening symbol to its corresponding closing pair in real time. Aside from that, despite the difficulty of the program being requested, the design ensures that the data is separated from the business logic, which means that the stack remains an abstract component, while the main program deals with parsing the strings, error handling, and matching rules. All of this is achieved through test runs given by the procedure we followed in ILO A and ILO B, even though we had to find a way to give way for the Supplementary activity as what was being requested was difficult. It definitely helped by giving us ample knowledge about C++ a little bit more.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:**
**ChatGPT**
**Gemini**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **AI assistance is used for code explanation and grammar only**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **AI was used to confirm if the given code will return the right output and to explain what the process was within the program for further understanding.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the

permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.